Code Analysis – Challenge Solution: Wonderland 2

By: Tehila Chananayev

Introduction

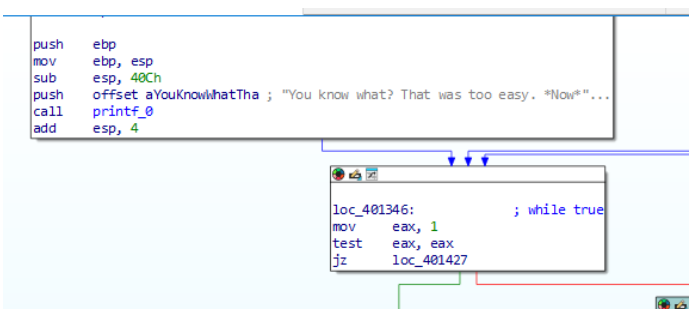
This challenge presents an Assembly code snippet in which the user inputs a string, and the program checks whether the input is correct. Our goal is to reverse-engineer the logic, identify the check condition, and discover the correct password that passes validation.

Code Structure

At the beginning of the code, the user is prompted to input a string via the scanf function, which is stored in memory. Immediately after, there's a while (true) loop initiated using:

test 1, 1

Since test 1, 1 always returns a non-zero result (sets ZF = 0), the condition is always true, creating an infinite loop — a classic way of implementing while (true) in Assembly.

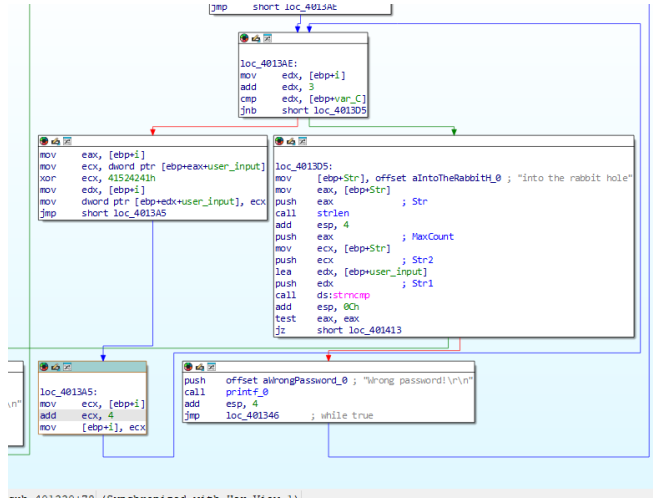


Key Variables

After this we see that there is a loop because there is a counter that is updated by 4 every time so I changed its name to I every 4 what happens? We will take the index that is, we have reached it we will add three to it and check if it is still less than **strlen** so we will change the name of var_C to user_input_length we will continue the process until we are 4 bytes away from the end of our string

- user_input: the string entered by the user.
- user_input_length: the length of the input

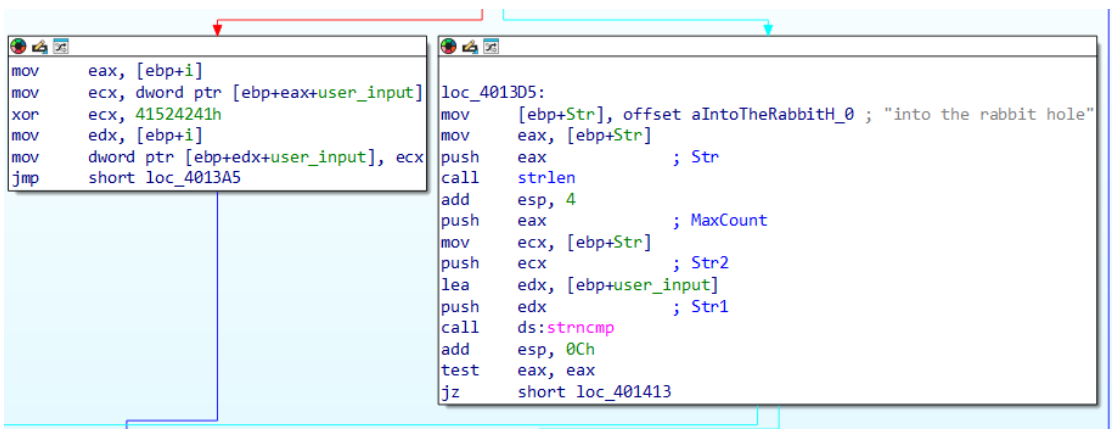
- i: a counter that increments by 4 in each iteration.



Core Logic

In each loop iteration:
1. Four bytes starting from user_input[i] are loaded into the ecx register.
2. A xor operation is applied with the value 0x41524241 ("ABRA" in ASCII).
3. The result of the XOR operation is compared with the target string.

This process continues chunk-by-chunk and aims to reconstruct the string: "into the rabbit hole".



Success Condition

If all XOR results from the input chunks match the target string "into the rabbit hole", the program prints the success message:
"Correct! You may enter..."

Python Script – Reverse XOR Solution

To find the correct input string, we reverse the XOR process by applying XOR between each 4-byte chunk of the target string and "ABRA":

```
def get_password():
    abra = b'ABRA'
```

```
    target = "into the rabbit hole"
    result = b""

    for i in range(0, len(target), 4):
        block = target[i:i+4].encode()
        xor_block = bytes([b ^ a for b, a in zip(block, abra)])
        result += xor_block

    return result.decode(errors='replace')
def main():
    password = get_password()
    print("Generated password:", password)

if __name__ == "__main__":
    main()
```

Expected Output:
Correct password: (,&.a6:$a03##+&a)->$

```
main
C:\Networks\Python3.8\python.exe C:/DBProject/main.py
Generated password: (,&.a6:$a03##+&a)->$

Process finished with exit code 0
```

Conclusion

The core mechanism of the validation is a XOR operation between the input string and "ABRA" repeated over 4-byte chunks. The result must match "into the rabbit hole". By reversing the XOR operation, we successfully extracted the required input.

Final password: (,&.a6:$a03##+&a)->$

```
Welcome to Wonderland. I am the mad hatter, and I have some riddles for you...
Input a level number (latest level- 2):
2


You know what? That was too easy. *Now* tell me the second password.
(Please enter the password)
Wrong password!
(,&.a6:$a03##+&a)->$
Correct! you may enter..
Welcome to Wonderland. I am the mad hatter, and I have some riddles for you...
Input a level number (latest level- 3):
```